
A05 State Machines: Pac-Man

Algorithmic Thinking
ALD (MTD.ba VZ)
SS26

Submission via Moodle

24 Points (+4 Bonus)

Pac-Man's ghosts are a classic example of a Finite State Machine. In this assignment you implement the Ghost AI as a Finite State Machine of your choice and wire the ghost into a simple maze world.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and explanations
 - Include all source files in a ZIP archive
 - Use the naming convention: `ALD_A05_LASTNAME.pdf` and `ALD_A05_LASTNAME.zip`
-

Exercise 1 (12 points) Ghost AI — Finite State Machine

A Pac-Man ghost has four states. It patrols its corner (SCATTER) until it spots Pac-Man, then chases (CHASE). When Pac-Man eats a power pellet the ghost panics (FRIGHTENED) and flees. If Pac-Man catches the ghost while it is frightened, it gets eaten and returns to base (EATEN).

The table below describes each state's expected behaviour:

State	Behaviour	Transition condition
SCATTER	Move toward assigned corner	$\text{distanceToPacMan}() < \text{detectionRange} \rightarrow \text{CHASE}$
CHASE	Move toward Pac-Man	$\text{distanceToPacMan}() \geq \text{detectionRange} \rightarrow \text{SCATTER}$
FRIGHTENED	Flee from Pac-Man; decrement ticks	$\text{distanceToPacMan}() \leq 1 \rightarrow \text{EATEN}; \text{ticks} \leq 0 \rightarrow \text{SCATTER}$
EATEN	Move toward ghost house base	at base $\rightarrow \text{SCATTER}$

Part A — FSM Design (3 points)

Design the ghost's Finite State Machine *before* you write any code. Draw a state diagram containing all four states and every transition. Label each arrow with the event or condition that triggers it. Include the diagram in your PDF (hand-drawn scan or digital tool both accepted). Note that a power pellet event must immediately switch the ghost to FRIGHTENED from *any* state, and unknown events leave the current state unchanged.

Part B — Implementation (6 points)

Implement the ghost AI as a Finite State Machine of your choice. Your implementation must correctly reproduce all four states and all transitions from your diagram. FRIGHTENED times out after a fixed number of ticks (default 8) and returns to SCATTER.

In your PDF: show your implementation and a console trace of the full SCATTER $\rightarrow$ CHASE $\rightarrow$ FRIGHTENED $\rightarrow$ EATEN $\rightarrow$ SCATTER cycle.

Part C — JUnit 5 Tests (3 points)

Include JUnit 5 tests covering all transitions, unknown-event self-loop, the power pellet trigger, FRIGHTENED timeout, and FRIGHTENED $\rightarrow$ EATEN. Show the `mvn test` output in your PDF.

Exercise 2 (12 points) Game World — Maze, Pac-Man, and Game Loop

Part A — Maze (3 points)

Implement a grid-based maze. The maze stores cells of four types: wall, dot, power pellet, and empty. Provide a hardcoded classic Pac-Man layout (minimum 8×8 , walls on all borders, at least 2 power pellets). Positions outside the grid boundaries count as walls.

In your PDF: print the maze to the console.

Part B — Pac-Man in the maze (5 points)

Implement Pac-Man navigating the maze. Pac-Man moves one cell per step and is silently blocked by walls. Eating a dot scores 10 points; eating a power pellet scores 50 points and immediately triggers the ghost's FRIGHTENED state. Pac-Man starts with 3 lives; running into a CHASE or SCATTER ghost costs one life. The ghost from Exercise 1 must be able to sense Pac-Man's position.

In your PDF: demonstrate Pac-Man eating a power pellet and the ghost entering FRIGHTENED.

Part C — Game Loop (4 points)

Implement a game loop that drives the simulation step by step: move Pac-Man, collect items, update the ghost, check for collision. Run a scripted demo of at least 10 steps that demonstrates a full power-pellet cycle: Pac-Man eats a power pellet, the ghost enters FRIGHTENED, several steps pass, and the ghost returns to SCATTER.

In your PDF: show the step logic and the full console output of your demo.

Bonus: JavaFX Visualization (+4 Points)

This section is entirely optional. The 4 bonus points are awarded on top of the 24 regular points.

Extend your solution with a JavaFX window that renders the maze and animates the ghost state changes:

- **Maze rendering:** Draw each cell as a fixed-size rectangle. Suggested colours: WALL = dark blue, DOT = small white circle, POWER_PELLET = large white circle, EMPTY = black background.
- **Pac-Man:** Yellow circle at the current grid position.
- **Ghost:** Coloured circle whose fill reflects the current state: SCATTER = red, CHASE = pink, FRIGHTENED = blue, EATEN = grey.
- **Animation:** Use an `AnimationTimer` to call `GameLoop.tick()` at approximately 3–5 fps. The ghost colour must visibly change during the FRIGHTENED cycle.

Add JavaFX as a dependency to your `pom.xml` (JavaFX 21, `javafx-controls` module). In your PDF: show the `AnimationTimer` setup and include a screenshot of the running visualization.